

Data Structures



Lecture 1

Linear Array Basics and Operations

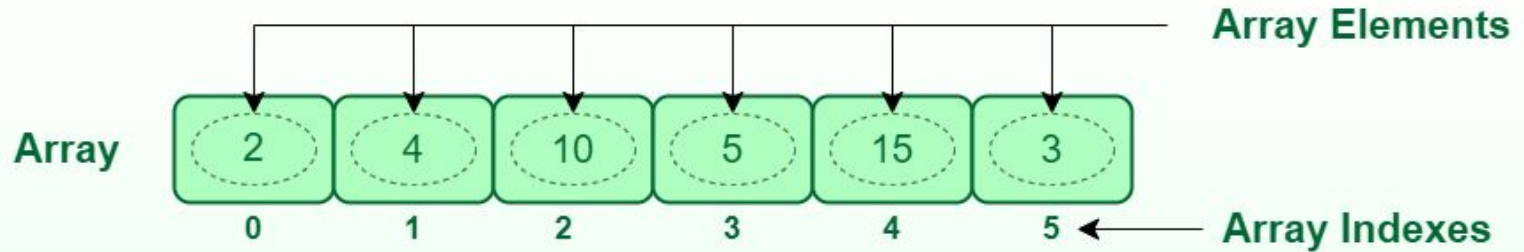
Prantik Paul [PNP]

Lecturer

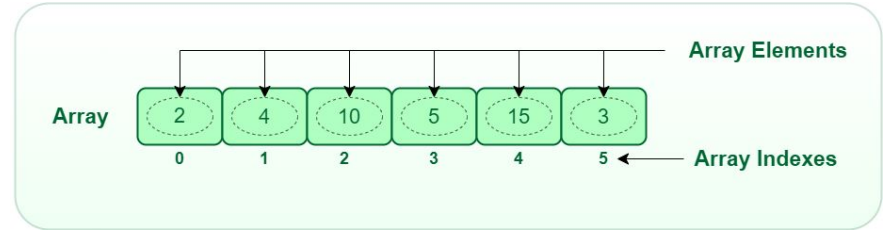
Department of Computer Science and Engineering

BRAC University

Array - Basics



Array - Basics



Dimension - fixed (1, 2, 3, ..., 100, ..)

Index

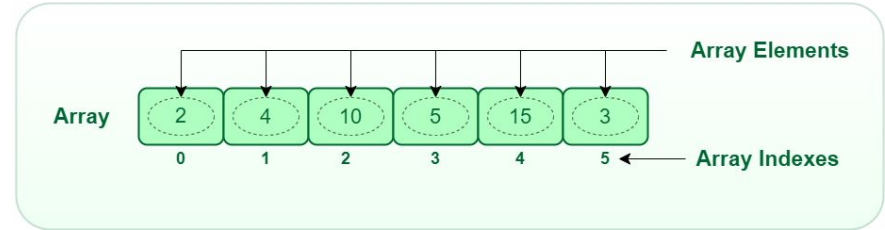
→ **0 - indexing**

→ **1 - indexing**

Size / Length

Type - Fixed

Array - Basics



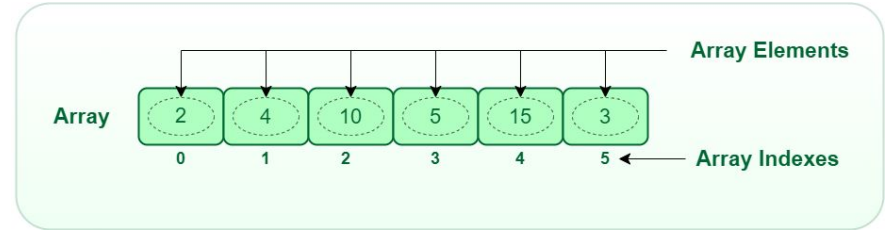
Dimension - fixed (1, 2, 3, ..., 100, ..)

Linear Arrays - 1D

Matrix - 2D

Tensor - 3D

Array - Basics



Index

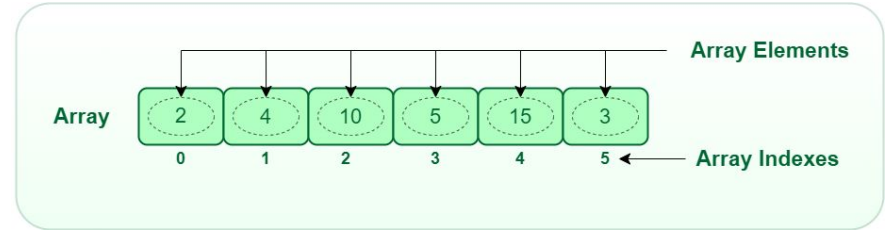
→ 0 - indexing

→ 1 - indexing

Why 0-indexing?

For efficient memory access

Array - Basics



Size / Length

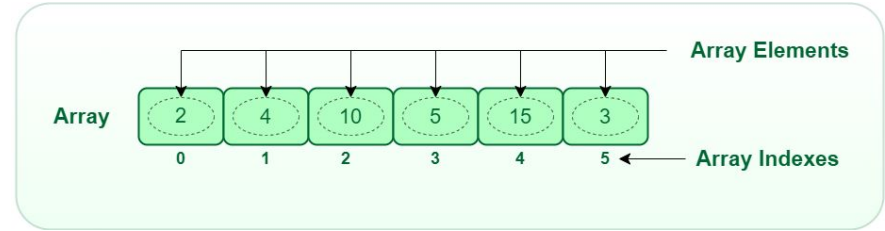
Size?

Number of Valid Elements

Length?

Fixed - As Declared

Array - Basics

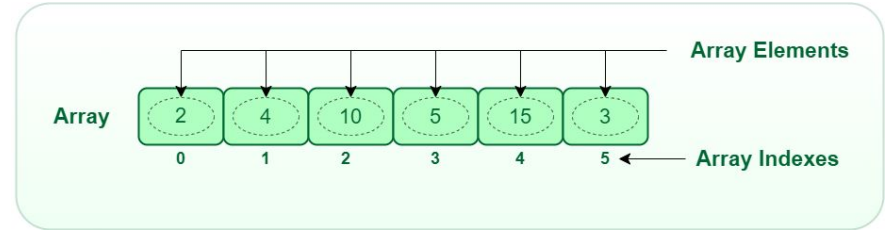


Type - Fixed

Why Not?

Efficient Memory Access

Array - Basics



Integer : 4bytes

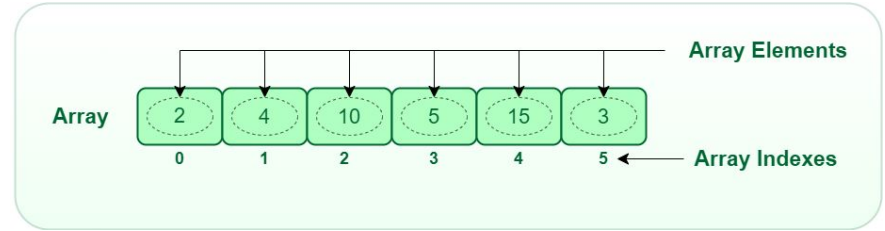
Floats : 8bytes

Character : 1byte

Formula-

**Starting address of array +
index*bytes required for one
data**

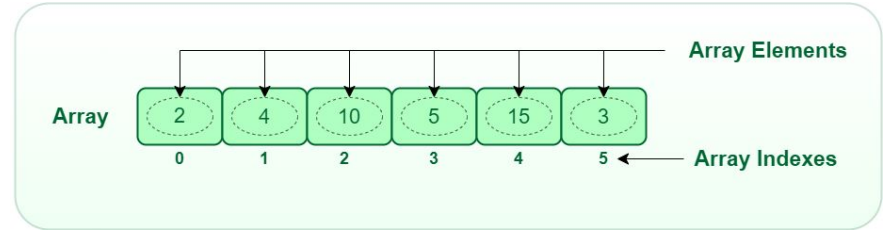
Array - Determine Length



Efficient-

- Doesn't store size
- May give invalid data if index out of bounds
- can access faster
- C Fortran

Array - Determine Length



Inefficient-

- Store size in first byte
- Check if index out of bound
- Slow access
- Java C#

Refresh your memory

1. `for i in range (stop)`
2. `for i in range (start, stop)`
3. `for i in range (start, stop, step)`
4. Negative step?
5. Negative stop?

Array - Accessing/Changing an Element

Suppose you have a 1D array named *student_names*. If you want get the 5th student from the array then you write:

```
fifth_student = student_names[5]
```

To change the value at 5th index and set it to 'John Doe', you write:

```
student_names[5] = 'John Doe'
```

Array - Initializing

```
array = [None] * length
```

Array - Initializing

```
Import numpy as np
```

```
my_array = np.array([8, 3, 13, 1])
```

Array - Initializing

```
Import numpy as np
```

```
my_list = [2, 45, 3, 2, 56]  
my_array = np.asarray(my_list)
```

Array - Initializing

```
Import numpy as np
```

```
size = 5  
my_array = np.zeros(size)
```

Array - Initializing

```
Import numpy as np
```

```
size = 5  
my_array = np.ones(size)
```

Array - Initializing

```
Import numpy as np
```

```
size = 5  
my_array = np.empty(size)
```

Array - Initializing

```
Import numpy as np
```

```
size = 5  
my_array = np.full(size, 4)
```

Array - Initializing

```
Import numpy as np
```

```
size = 5  
my_array = np.ones(size)  
print(my_array.size)
```

Array - Initializing

```
Import numpy as np
```

```
size = 5  
my_array = np.ones(size)  
print(len(my_array))
```

Array - Functions (Iteration)

```
def iteration(source):  
    for i in range(len(source)):  
        print(source[i])
```

Array - Functions (Reverse Iteration)

```
def reverseIteration(source):  
    for i in range(len(source) - 1, -1, -1):  
        print(source[i])
```

Array - Functions (Copy Array)

```
1. FUNCTION copy_array(arr)
2.     arr2 = create_array(size of arr)
3.     FOR i = 0 TO size of arr - 1
4.         arr2[i] = arr[i]
5.     END FOR
6.     RETURN arr2
7. END FUNCTION
```

Array - Functions (Copy Array)

```
def copyArray(source):  
    newArray= np.array([0]* len(source))  
    for i in range(len(source)):  
        newArray[i]= source[i]  
    return newArray
```

Array - Functions (Resize Array)

1. function `resize(arr, new_size)`
2. set `arr2` to an array of size `new_size`, initialized with 0
3. set `i` to 0
4. while `i` is less than length of `arr`
5. set `arr2[i]` to `arr[i]`
6. increment `i` by 1
7. end while
8. return `arr2`
9. end function

Array - Functions (Resize Array)

```
def resizeArray(oldArray, newCapacity):  
    newArray= np.array([0]* newCapacity)  
    for i in range(len(oldArray)):  
        newArray[i]= oldArray[i]  
    return newArray
```

Array - Functions (Reverse Array)

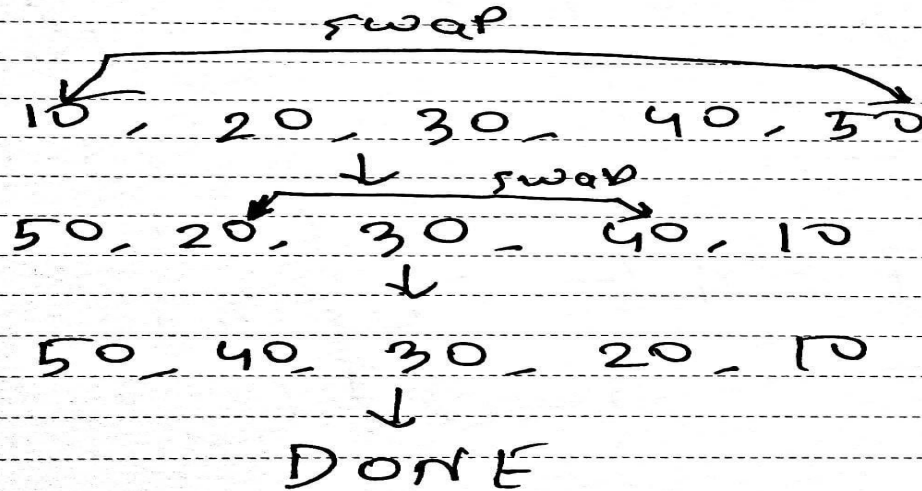
```
1. FUNCTION reverse_out_of_place(arr)
2.     arr2 = create_array(size of arr)
3.     i = 0
4.     j = size of arr - 1
5.     WHILE i <= size of arr - 1
6.         arr2[i] = arr[j]
7.         i = i + 1
8.         j = j - 1
9.     END WHILE
10.    RETURN arr2
11. END FUNCTION
```

Array - Functions (Reverse Array)

```
def reverse_out_of_place(source):  
    reversed= np.zeros(len(source), dtype=int)  
    i= 0  
    while(i<=len(source)-1):  
        reversed[i]= source[len(source)-1-i]  
        i+=1  
    return reversed
```

Array - Functions (Reverse Array) (Efficient)

In place Reverse:



Array - Functions (Reverse Array) (Efficient)

```
1. FUNCTION reverse_in_place(arr)
2.     j = size of arr - 1
3.     FOR i = 0 TO (size of arr - 1) DIVIDED BY 2
4.         SWAP arr[i] WITH arr[j]
5.         j = j - 1
6.     END FOR
7.     RETURN arr
8. END FUNCTION
```

Array - Functions (Reverse Array) (Efficient)

```
def revArrInPlace(arr):  
    i = 0  
    j = len(arr) - 1  
    while i < j:  
        temp = arr[i]  
        arr[i] = arr[j]  
        arr[j] = temp  
        i += 1  
        j -= 1
```

Array - Functions (Shift Left)

```
1. FUNCTION shift_left(arr)
2.     FOR i = 1 TO size of arr - 1
3.         arr[i-1] = arr[i]
4.     END FOR
5.     arr[size of arr - 1] = 0
6.     RETURN arr
7. END FUNCTION
```

Array - Functions (Shift Left)

```
def shiftLeft(arr):  
    for i in range(1, len(arr)):  
        arr[i-1] = arr[i]  
    arr[len(arr) - 1] = None  
    return arr
```

Array - Functions (Shift Right)

```
1. FUNCTION shift_right(arr)
2.     FOR i = size of arr - 1 DOWNTO 1
3.         arr[i] = arr[i-1]
4.     END FOR
5.     arr[0] = 0
6.     RETURN arr
7. END FUNCTION
```

Array - Functions (Shift Right)

```
def shiftRight(arr):  
    for i in range(len(arr) - 1, 0, -1):  
        arr[i] = arr[i - 1]  
    arr[0] = None  
    return arr
```

Array - Functions (Rotate Left)

```
1. FUNCTION rotate_left(arr)
2.     temp = arr[0]
3.     FOR i = 1 TO size of arr - 1
4.         arr[i-1] = arr[i]
5.     END FOR
6.     arr[size of arr - 1] = temp
7.     RETURN arr
8. END FUNCTION
```

Array - Functions (Rotate Left)

```
def rotateLeft(arr):  
    temp = arr[0]  
    for i in range(1, len(arr)):  
        arr[i-1] = arr[i]  
    arr[len(arr) - 1] = temp  
    return arr
```

Array - Functions (Rotate Right)

```
1. FUNCTION rotate_right(arr)
2.     temp = arr[size of arr - 1]
3.     FOR i = size of arr - 1 DOWNTO 1
4.         arr[i] = arr[i-1]
5.     END FOR
6.     arr[0] = temp
7.     RETURN arr
8. END FUNCTION
```

Array - Functions (Rotate Right)

```
def rotateRight(arr):  
    temp = arr[len(arr) - 1]  
    for i in range(len(arr) - 1, 0, -1):  
        arr[i] = arr[i - 1]  
    arr[0] = temp  
    return arr
```

Array - Functions (Insert)

#Inserting anywhere

1. FUNCTION insert_anywhere(arr, size, index, elem)
2. IF index < 0 OR index > size
3. RETURN "Insertion Not Possible"
4. END IF
5. IF size >= size of arr
6. arr = resize_array(arr, size of arr + 3)
7. END IF
8. FOR i = size DOWNT0 index + 1
9. arr[i] = arr[i-1]
10. END FOR
11. arr[index] = elem
12. RETURN arr
13. END FUNCTION

Array - Functions (Insert)

```
def insertElement(arr, size, elem, index):  
    # Practice how to throw exception if there is no empty space  
    if size == len(arr):  
        print("No space left. Insertion failed")  
    else:  
        for i in range(size, index, -1):  
            arr[i] = arr[i - 1] #Shifting right till the index  
        arr[index] = elem #Inserting element  
    return arr
```

Array - Functions (Remove)

#Deleting any element

1. FUNCTION delete_any_element(arr, size, index)
2. IF size = 0 OR (index < 0 AND index >= size)
3. RETURN "Deletion Not Possible"
4. END IF
5. FOR i = index TO size - 1
6. arr[i] = arr[i+1]
7. END FOR
8. arr[size-1] = 0
9. RETURN arr
10. END FUNCTION

Array - Functions (Remove)

```
def removeElement(arr, index, size):  
    for i in range(index + 1, size):  
        arr[i - 1] = arr[i] #Shifting left from removing index  
    arr[size - 1] = None #Making last space empty
```

Array - Selection Sort

```
selectionSort(array, size)
  repeat (size - 1) times
    set the first unsorted element as the minimum
    for each of the unsorted elements
      if element < currentMinimum
        set element as new minimum
    swap minimum with first unsorted position
  end selectionSort
```

Array - Selection Sort

```
def selectionSort(array, size):  
  
    for step in range(size):  
        min_idx = step  
  
        for i in range(step + 1, size):  
  
            # select the minimum element in each loop  
            if array[i] < array[min_idx]:  
                min_idx = i  
  
        # put min at the correct position  
        (array[step], array[min_idx]) = (array[min_idx], array[step])
```

Array - Bubble Sort

SELF STUDY

Array - Binary Search

```
do until the pointers low and high meet each other.  
    mid = (low + high)/2  
    if (x == arr[mid])  
        return mid  
    else if (x > arr[mid]) // x is on the right side  
        low = mid + 1  
    else // x is on the left side  
        high = mid - 1
```

Array - Binary Search

```
def binarySearch(array, x, low, high):  
  
    # Repeat until the pointers low and high meet each other  
    while low <= high:  
        mid = low + (high - low)//2  
  
        if array[mid] == x:  
            return mid  
  
        elif array[mid] < x:  
            low = mid + 1  
  
        else:  
            high = mid - 1  
  
    return -1
```